

Implementación de pipelines CI/CD, devSecOps y observabilidad

Jhonatan Winston Sotelo
Carlos Arturo Salazar Castaneda
Juan Sebastian Buitrago Romero
Santiago Eduardo Muñoz Castillo
Agosto 2026.

Universidad de la Sabana
Fundamentos de DevOps

Tabla de Contenidos

Resumen Ejecutivo	1
Descripción detallada del flujo CI/CD.....	2
Flujo de Integración Continua (CI) en GitHub Actions.....	2
Flujo de Despliegue Continuo (CD) en Jenkins	2
Herramientas utilizadas y justificación	4
Evidencias técnicas de seguridad y monitoreo	6
Integración y Evidencias de Seguridad (Snyk & SonarQube).....	6
Monitoreo Activo (Prometheus y Grafana)	8
Archivos de Configuración de Orquestación e Infraestructura.....	14
Reflexión sobre eficiencia operativa.....	15
Referencias.....	16

Resumen Ejecutivo

Este documento recopila la sustentación técnica y el flujo diseñado para el laboratorio técnico de la Unidad 3. Se presenta una implementación funcional de un pipeline de Integración Continua (CI) en GitHub Actions y de Despliegue Continuo (CD) en Jenkins. El proyecto adopta la filosofía DevSecOps al incorporar análisis estático de código con SonarQube y escaneo de vulnerabilidades en dependencias con Snyk. Asimismo, se habilita la observabilidad en caliente mediante el despliegue de contenedores de Prometheus y Grafana, garantizando un flujo robusto, automatizado y monitoreado bajo estándares de la industria.

Descripción detallada del flujo CI/CD

La arquitectura de entrega automatizada del laboratorio se estructura de manera híbrida, dividiendo las etapas entre procesos locales ligeros de validación en la nube (CI) y tareas robustas de empaquetado y despliegue físico en el servidor de destino (CD). Esta separación maximiza el aprovechamiento de los recursos de cómputo y establece un proceso riguroso de control de calidad previo a la puesta en marcha de la aplicación.

Flujo de Integración Continua (CI) en GitHub Actions

El pipeline de CI se ejecuta de forma automática ante cualquier push o pull request hacia las ramas main o develop.

Actúa como el primer filtro de seguridad y calidad del código, compuesto por los siguientes pasos secuenciales:

- **Checkout del código:** Descarga los archivos fuentes del repositorio mediante la acción `actions/checkout@v4`.
- **Configuración de Node.js:** Inicializa un entorno de ejecución Node.js versión 20.
- **Instalación de dependencias:** Ejecuta `npm install` para descargar las librerías requeridas.
- **Análisis estático (ESLint):** Valida las normas de estilo y mejores prácticas sintácticas definidas en el proyecto.
- **Pruebas de Software:** Ejecuta el set de pruebas unitarias implementado con Jest y genera métricas de cobertura de código.
- **Análisis de dependencias (Snyk):** Analiza el manifiesto `package.json` contra bases de datos globales de vulnerabilidades (SCA).
- **Escaneo de código (SonarQube):** Corre el analizador de SonarQube para identificar problemas estructurales, bugs latentes y duplicados.
- **Construcción de imagen:** Valida de forma simulada la compilación de la imagen utilizando el Dockerfile nativo para asegurar su funcionalidad.

Flujo de Despliegue Continuo (CD) en Jenkins

Una vez el código supera de forma exitosa las comprobaciones automáticas de GitHub, se activa el proceso de Despliegue Continuo gestionado por Jenkins. Este pipeline está estructurado de manera declarativa dentro del archivo `Jenkinsfile` y corre de forma automatizada sobre un servidor EC2 dedicado, orquestando las etapas operativas:

- **Clonación del repositorio:** Jenkins descarga el código limpio validado utilizando parámetros configurables del Job.
- **Verificación de Docker:** Valida que el Docker Engine se encuentre activo en el servidor anfitrión.
- **Inyección de Herramientas:** Configura Node.js dinámicamente utilizando el plugin NodeJS del servidor Jenkins.
- **Escaneo de dependencias (Snyk):** Ejecuta `npx snyk test` inyectando la credencial `SNYK_TOKEN`.
- **Análisis de Calidad (SonarQube):** Ejecuta el análisis estático en caliente usando la herramienta SonarScanner.
- **Quality Gate Webhook:** Pausa la ejecución mediante `waitForQualityGate`. El pipeline queda en espera de una confirmación HTTP asíncrona enviada por el webhook de SonarQube. Si el Quality Gate es fallido, el pipeline se aborta inmediatamente, protegiendo el entorno de despliegue de versiones defectuosas.
- **Construcción de imagen Docker:** Compila la imagen de la aplicación utilizando el número secuencial de Jenkins (`BUILD_NUMBER`) e inyecta el tag de última versión (`latest`).
- **Limpieza de contenedores:** Detiene y elimina de forma automática cualquier contenedor anterior que use el puerto 3000 evitando colisiones o fallas del demonio de Docker.
- **Despliegue:** Arranca el nuevo contenedor en segundo plano exponiendo el puerto 3000.
- **Smoke Test:** Realiza una pausa técnica de 5 segundos y ejecuta un comando `curl` hacia el endpoint `http://localhost:3000/health`. El pipeline confirma el despliegue exitoso si recibe un código de respuesta HTTP exitoso y la estructura JSON `{"status":"UP"}`.

Herramientas utilizadas y justificación

A continuación, se presenta un cuadro comparativo detallado que resume las herramientas implementadas en la arquitectura DevOps del laboratorio técnico, su función específica y su justificación técnica en el ciclo de vida del software:

Herramienta	Función DevOps	Justificación Técnica
GitHub Actions	Integración Continua (CI)	Ejecución ágil, descentralizada y nativa en la nube. Permite validar errores sintácticos y pruebas unitarias de inmediato, ahorrando recursos de servidor local.
Jenkins	Despliegue Continuo (CD)	Orquestador extensible de nivel empresarial. Permite administrar de forma centralizada credenciales seguras, automatizar tareas en el servidor anfitrión y conectar webhooks de control.
Docker	Contenerización	Garantiza ambientes de ejecución aislados y consistentes entre el desarrollo local y el servidor, eliminando fallos por diferencias de sistema operativo o dependencias del host.
SonarQube	Análisis de Calidad	Supervisa de forma integral la calidad técnica del software, detecta vulnerabilidades, bugs estructurales y bloquea despliegues deficientes mediante Quality Gates.
Snyk	Seguridad en Librerías	Escanea las dependencias directas e indirectas del proyecto buscando fallas conocidas de seguridad, protegiendo la aplicación de vulnerabilidades de terceros en tiempo real.
Prometheus	Recolección y almacenamiento de métricas	Recopila y almacena métricas operativas de la infraestructura y de los contenedores mediante un modelo de series temporales. Permite consultar indicadores como disponibilidad, consumo de recursos y estado de los servicios, proporcionando la información necesaria para identificar comportamientos anómalos y soportar la generación de alertas.
cAdvisor	Recolección de métricas de los contenedores Docker	Recopila métricas de rendimiento y consumo de recursos de los contenedores Docker, incluyendo CPU, memoria, red y otros indicadores operativos. Se encuentra conectado al entorno Docker del servidor para proporcionar a

		Prometheus información detallada sobre el comportamiento de los contenedores.
Grafana	Visualización, análisis y generación de alertas	Proporciona una capa de visualización y observabilidad sobre las métricas recopiladas por Prometheus. Permite construir dashboards interactivos para supervisar el estado de la aplicación y la infraestructura, identificar tendencias y establecer reglas de alerta ante condiciones anómalas, facilitando una respuesta temprana frente a posibles incidentes
Gmail/SMTP	Canal de notificación de alertas	Se utiliza como canal de notificación para las alertas generadas por Grafana. La integración mediante SMTP permite enviar automáticamente mensajes al correo configurado cuando una regla de alerta cambia a un estado de firing , facilitando la comunicación de incidentes sin necesidad de consultar continuamente el dashboard.

Evidencias técnicas de seguridad y monitoreo

Integración y Evidencias de Seguridad (Snyk & SonarQube)

La integración de las herramientas de seguridad se realizó siguiendo rigurosos estándares. A continuación, se describen las configuraciones y los resultados validados en el laboratorio:

- **Credenciales Seguras:** Las variables de acceso altamente sensibles, tales como `SNYK_TOKEN` y `SONAR_TOKEN`, se registraron de forma encriptada bajo la bóveda de seguridad de GitHub (en *Settings > Secrets and Variables > Actions*) y en el almacén de credenciales seguras de Jenkins en formato **Secret Text**. Esto evita la fuga accidental de secretos en la consola o en el código fuente.
- **SonarQube Server & Webhook:** El servidor de análisis estático se encuentra corriendo en una instancia de contenedor segura (puerto 9000). Para automatizar la sincronización y la etapa 'SonarQube Quality Gate', se dio de alta un webhook HTTP global dentro de SonarQube apuntando a la dirección IP del servidor Jenkins con la ruta `/sonarqube-webhook/`. Esto permite que el pipeline de Jenkins reciba instantáneamente el veredicto del Quality Gate.
- **Análisis de Snyk en Dependencias:** La herramienta se integró de forma declarativa mediante NPM y ejecutores Snyk en los flujos de CI/CD. El comando `snyk test --severity-threshold=high` analiza las dependencias y reporta en consola los fallos detectados de prioridad crítica o alta. Adicionalmente, el repositorio se importó en el dashboard web de *Snyk.io*, permitiendo un seguimiento interactivo de las dependencias vulnerables y parches aplicables.

Dentro del Análisis de dependencias que hizo Snyk no encontró ninguna vulnerabilidad para este caso

```
1 ▶ Run snyk/actions/node@master
8 /usr/bin/docker run --name snyksnyknode_12972d --label ddac51 --workdir /github/workspace --rm -e "SNYK_TOKEN" -e "INPUT_ARGS" -e "INPUT_COMMAND" -e "INPUT_JSON" -e "FORCE_COLOR" -e
"SNYK_INTEGRATION_NAME" -e "SNYK_INTEGRATION_VERSION" -e "HOME" -e "GITHUB_JOB" -e "GITHUB_REF" -e "GITHUB_SHA" -e "GITHUB_REPOSITORY" -e "GITHUB_REPOSITORY_OWNER" -e
"GITHUB_REPOSITORY_OWNER_ID" -e "GITHUB_RUN_ID" -e "GITHUB_RUN_NUMBER" -e "GITHUB_RETENTION_DAYS" -e "GITHUB_RUN_ATTEMPT" -e "GITHUB_ACTOR_ID" -e "GITHUB_ACTOR" -e "GITHUB_WORKFLOW" -e
"GITHUB_HEAD_REF" -e "GITHUB_BASE_REF" -e "GITHUB_EVENT_NAME" -e "GITHUB_SERVER_URL" -e "GITHUB_API_URL" -e "GITHUB_GRAPHQL_URL" -e "GITHUB_REF_NAME" -e "GITHUB_REF_PROTECTED" -e
"GITHUB_REF_TYPE" -e "GITHUB_WORKFLOW_REF" -e "GITHUB_WORKFLOW_SHA" -e "GITHUB_REPOSITORY_ID" -e "GITHUB_TRIGGERING_ACTOR" -e "GITHUB_WORKSPACE" -e "GITHUB_ACTION" -e "GITHUB_EVENT_PATH" -e
"GITHUB_ACTION_REPOSITORY" -e "GITHUB_ACTION_REF" -e "GITHUB_PATH" -e "GITHUB_ENV" -e "GITHUB_STEP_SUMMARY" -e "GITHUB_STATE" -e "GITHUB_OUTPUT" -e "GITHUB_ARTIFACTS" -e
"GITHUB_ARTIFACTS_LIST" -e "RUNNER_OS" -e "RUNNER_ARCH" -e "RUNNER_NAME" -e "RUNNER_ENVIRONMENT" -e "RUNNER_TOOL_CACHE" -e "RUNNER_TEMP" -e "RUNNER_WORKSPACE" -e "ACTIONS_RUNTIME_URL" -e
"ACTIONS_RUNTIME_TOKEN" -e "ACTIONS_CACHE_URL" -e "ACTIONS_RESULTS_URL" -e "ACTIONS_ORCHESTRATION_ID" -e "GITHUB_ACTIONS=true" -e "CI=true" -v "/var/run/docker.sock":"/var/run/docker.sock" -v
"/home/runner/work/_temp":"/github/runner_temp" -v "/home/runner/work/_temp/github_home":"/github/home" -v "/home/runner/work/_temp/github_workflow":"/github/workflow" -v
"/home/runner/work/_temp/runner_file_commands":"/github/file_commands" -v "/home/runner/work/CICD-jenkins-securityAndMonitoring/CICD-jenkins-securityAndMonitoring":"/github/workspace"
snyk/snyk:node "snyk" "test" "--severity-threshold=high"
9
10 Testing /github/workspace...
11
12 Organization: caarsaca15
13 Package manager: npm
14 Target file: package-lock.json
15 Project name: cicd-lab-webapp
16 Open source: no
17 Project path: /github/workspace
18 Licenses: enabled
19
20 ✓ Tested 68 dependencies for known issues, no vulnerable paths found.
```


Pero revisando el Dashboard nos encontramos con una vulnerabilidad en cuanto a la versión de Node Js que manejamos para construir la imagen en este ejercicio

caarsaca15 > Projects > winstonjs/CICD-jenkins-securityAndMonitoring main

Dockerfile

Overview History Settings

+ Add a value + Add a value + Add a value + Add a value

Recommendations for upgrading the base image

	BASE IMAGE	VULNERABILITIES	SEVERITY
Current image	node:20-alpine	25	0 C 0 H 0 M 25 L
Major upgrades	node:26.7.0-alpine	10	0 C 0 H 0 M 10 L

[View docs](#) [Open a fix PR](#)

Nos recomienda actualizar la versión a la 26.7 ya que contiene menos vulnerabilidades actualmente que la versión 20 pasando de 25 a 10 vulnerabilidades

- **Análisis Estático:** El reporte se genera automáticamente en cada ejecución del pipeline y se aloja en el dashboard de SonarQube, detallando el número exacto de bugs, code smells detectados y vulnerabilidades críticas del código fuente.

Dentro del análisis de seguridad y calidad del Código, Sonar nos dio la siguiente calificación:

CICD-jenkins-securityAndMonitoring Public

Last analysis: 8 minutes ago • 19 Lines of Code • JavaScript

Passed

1	0	0	—	0.0%	0.0%
Security	Reliability	Maintainability	Hotspots Reviewed	Coverage	Duplications

1 of 1 shown

De momento tenemos casi todas las variables en calificación A, menos la de seguridad la cual está en B. En este momento está reportando un incidente de magnitud baja y es el siguiente:

The screenshot shows a SonarQube issue report for a security vulnerability. The issue title is "This framework implicitly discloses version information by default. Make sure it is safe here." and it is categorized as a security issue with a severity of "Low". The issue is located in a web application technology and is associated with the JavaScript file `javascript:S5689`. The issue is currently "Open" and "Not assigned". The issue is caused by a "cwe" (Common Weakness Enumeration) and is "Introduced: 18 hours ago". The issue is associated with the "Software qualities impacted" (Security) and the "Code attribute" (Responsibility | Not trustworthy). The issue is located in the file `125617... jhonat...` and is associated with the code snippet:

```
4  const PORT = process.env.PORT || 3000;
5
6  app.get("/", (req, res) => {
7    res.status(200).json({
8      application: "cicd-lab-securty-monitoring-webapp",
9      status: "ok",
10     message: "CI/CD laboratory application 2"
11   });
12 });
```

Esto significa que Express puede revelar información sobre su versión mediante ciertos headers HTTP, y Sonar considera que eso facilita el fingerprinting de la aplicación.

Monitoreo Activo (Prometheus y Grafana)

Integración y funcionamiento del sistema de monitoreo activo implementado para la infraestructura CI/CD y aplicaciones en contenedores. El entorno combina Prometheus, Grafana, cAdvisor y Jenkins junto con un sistema de alertas vía correo electrónico/notificaciones.

El sistema de monitoreo está diseñado para ofrecer visibilidad en tiempo real del estado de la infraestructura, métricas de rendimiento y alertas proactivas ante fallos de servicio.

La **infraestructura** base se ejecuta sobre un entorno Dockerizado con la siguiente distribución de servicios:

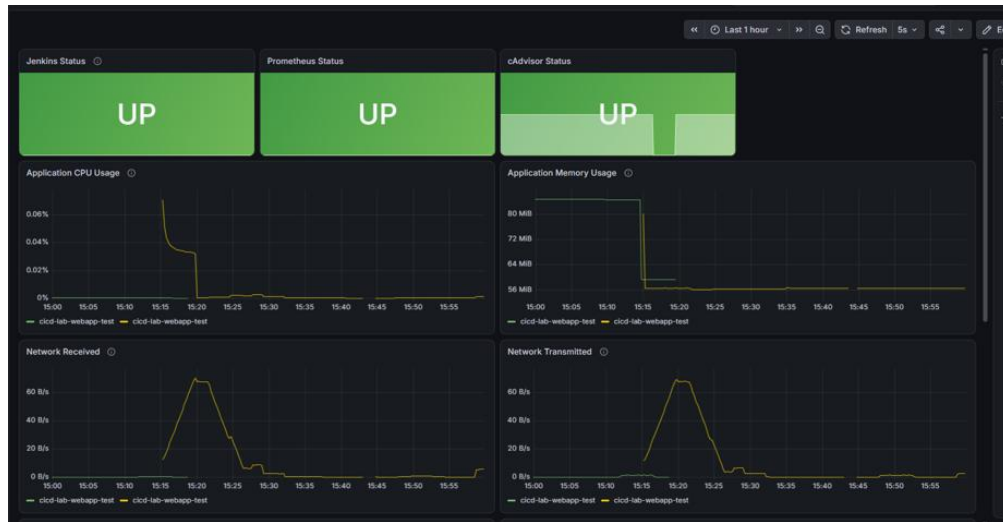
Contenedor / Nombre	Imagen	Puertos Mapeados	Función en el Ecosistema
grafana	grafana/grafana	0.0.0.0:3002->3000/tcp	Visualización de Dashboards y motor de Alertas.
prometheus	prom/prometheus	0.0.0.0:9090->9090/tcp	Recolección y almacenamiento de series temporales.
cadvisor	gcr.io/cadvisor/cadvisor:latest	0.0.0.0:8081->8080/tcp	Monitoreo de métricas en contenedores Docker.
cicd-lab-webapp-test	cicd-lab-webapp-test:latest	0.0.0.0:3000->3000/tcp	Aplicación objetivo sometida a pruebas y monitoreo.

El Dashboard de Grafana está estructurado en tres secciones clave:

A. Estado General de los Servicios (Health Check Panel)

Paneles de estado rápido en la parte superior que muestran el estado de disponibilidad (UP/DOWN) de:

- **Jenkins Status:** UP (monitoreado vía métricas exportadas desde la JVM/Host).
- **Prometheus Status:** UP.
- **cAdvisor Status:** UP.



B. Rendimiento de la Aplicación y Red

- **Application CPU Usage & Memory Usage:** Gráficas temporales que registran el consumo de la aplicación cicd-lab-webapp-test.
- **Network Received / Transmitted:** Seguimiento del ancho de banda y tráfico de red entrante/saliente consumido por los contenedores.

C. Rendimiento de la JVM en Jenkins y Contenedores Globales

- **Jenkins JVM Memory:** Gráfica comparativa entre el límite máximo de memoria Heap (Heap Max ~2 GiB) y el uso dinámico real (Heap Used).
- **Jenkins JVM CPU:** Porcentaje de CPU asignado a los procesos de ejecución de builds en Jenkins.
- **All Docker Containers CPU & Memory:** Vista consolidada del uso de recursos de todos los servicios activos (SonarQube, Grafana, Prometheus, cAdvisor, WebApp).

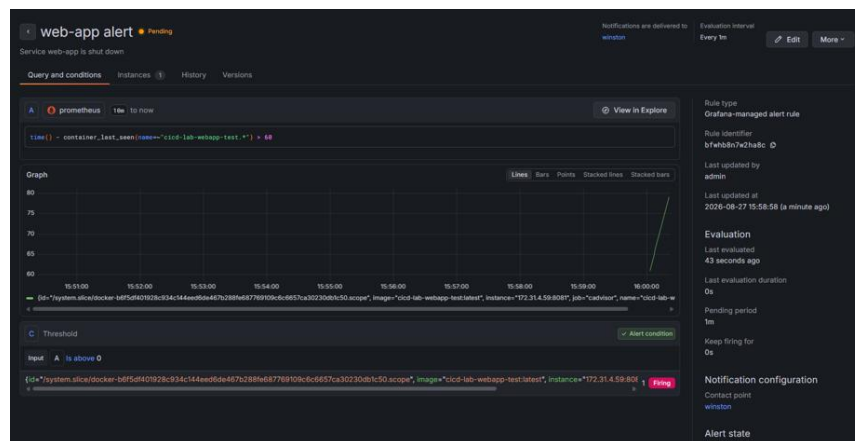


Flujo y Configuración de Alertas (Alerting System)

Grafana cuenta con un motor de reglas administradas (Grafana-managed alert rules) configurado para detectar la caída de servicios críticos.

Regla de Alerta: web-app alert

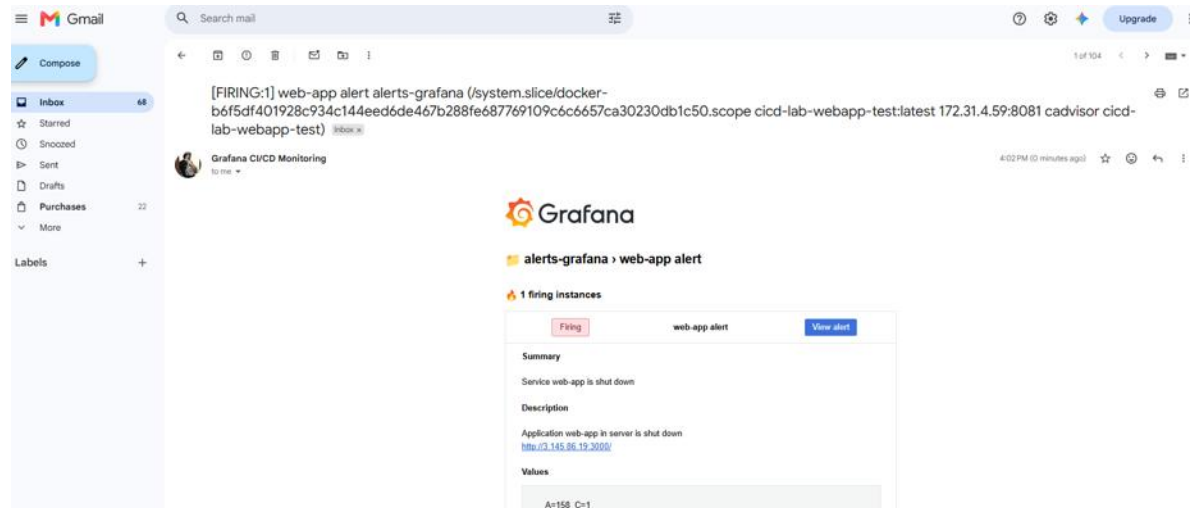
- **Métrica evaluada:** `time() - container_last_seen{name=~"cicd-lab-webapp-test.*"} > 60`
- **Condición de disparo:** Si el contenedor de la aplicación no reporta actividad o señal de vida en cAdvisor durante más de 60 segundos, la regla pasa a estado **Pending** y posteriormente **Firing**.
- **Acción / Notificación:** Grafana activa el punto de contacto (Contact Point) configurado (ej. winston) para despachar inmediatamente una notificación electrónica.



Flujo del Ciclo de Vida de la Alerta:

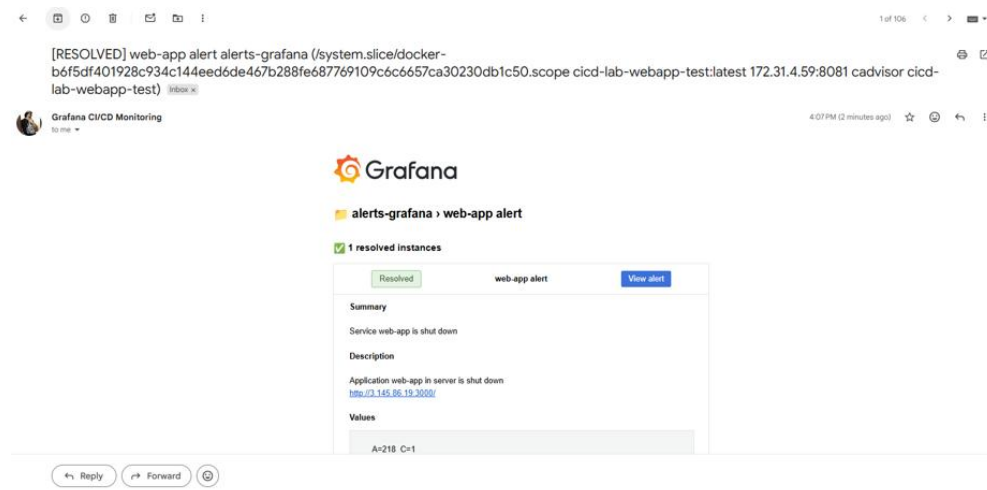
1. **Disparo de Alerta ([FIRING]):** Al apagar o fallar el contenedor objetivo, Grafana envía una notificación con el asunto:

[FIRING:1] web-app alert alerts-grafana (cicd-lab-webapp-test) El correo detalla que el servicio *"Application web-app in server is shut down"*.



2. **Recuperación de Alerta ([RESOLVED]):** Una vez que el contenedor vuelve a estar activo y cAdvisor vuelve a reportar métricas a Prometheus, Grafana procesa la recuperación y envía automáticamente un mensaje de cierre:

[RESOLVED] web-app alert alerts-grafana (cicd-lab-webapp-test) Esto confirma que la incidencia ha sido resuelta y el sistema se encuentra en estado saludable.



Archivos de Configuración de Orquestación e Infraestructura

Alineado con los requisitos del laboratorio técnico para la Unidad 3, se incorporaron en el repositorio los archivos de configuración para el despliegue en Kubernetes de manera declarativa (Infraestructura como Código - IaC). Esto incluye el archivo `k8s/deployment.yml` (el cual define la réplica de 2 Pods para alta disponibilidad de la aplicación con límites estrictos de CPU y memoria) y el archivo `k8s/service.yml` (que expone la aplicación de forma pública utilizando el puerto 32000 mediante un servicio de tipo NodePort).

Reflexión sobre eficiencia operativa

La implementación completa de este flujo automatizado con controles de seguridad e integración de monitoreo proporciona una perspectiva clara sobre el valor estratégico de la cultura DevOps en entornos de desarrollo modernos. A continuación, se detallan las principales reflexiones sobre la eficiencia operativa del sistema:

- **Filosofía 'Shift-Left':** Tradicionalmente, la seguridad de las aplicaciones se evaluaba justo antes de la puesta en producción o de forma periódica mediante auditorías externas. En esta arquitectura, herramientas como Snyk y SonarQube analizan de forma continua cada cambio de código. Esto reduce los costos de remediación de fallas drásticamente, pues es significativamente más rápido y económico corregir un bug sintáctico o una dependencia vulnerable en la máquina del desarrollador que hacerlo tras un incidente de seguridad en entornos productivos.
- **Automatización de Tareas Repetitivas:** Tareas operativas que solían requerir la intervención manual del equipo de sistemas, como detener contenedores obsoletos, validar puertos libres en el host o realizar pruebas de humo del servicio, ahora ocurren en milisegundos de forma programática. La eliminación de comandos manuales como `docker stop` o `docker rm` reduce drásticamente la tasa de error humano y agiliza las ventanas de lanzamiento.
- **Protección del Entorno mediante Quality Gates:** La configuración del webhook que conecta a SonarQube con Jenkins garantiza que solo aquellas versiones de software que superan las métricas mínimas de calidad y seguridad de la organización sean desplegadas. Esto actúa como un guardián de calidad automatizado, impidiendo de forma robusta regresiones de código y mejorando sustancialmente la estabilidad operativa general del sistema.
- **Observabilidad de Reactiva a Proactiva:** La integración de Prometheus y Grafana redefine el soporte operativo. En lugar de esperar la notificación o la queja de un cliente externo alertando de una falla o lentitud, el equipo técnico cuenta con una visualización de rendimiento en tiempo real que le permite identificar fugas de memoria o saturación de CPU de forma proactiva, garantizando la resiliencia técnica de la infraestructura.

Referencias

<https://gist.github.com/darinpope/1c8422fb7512411760ccb2827d82613f>